

Problem 2: OpenMP Acceleration and Correctness Test for the NEP Force Path

May 23, 2026

1 Goal

Problem 2 requires OpenMP acceleration for molecular-dynamics potential calculation, with special attention to data dependencies, reductions, and correctness under different thread counts. In this work, the target is the NEP CPU implementation under `source/source_md/potential/ml/nep`.

2 Implementation

The descriptor and neighbor-list construction paths already contained OpenMP parallel loops. The remaining risky hotspot was the force path:

- `find_force_radial_small_box`
- `find_force_angular_small_box`
- `find_force_ZBL_small_box`

These loops cannot be parallelized safely with a plain `#pragma omp parallel for`, because each neighbor contribution updates both atoms in a pair:

$$F_{n1} += f_{12}, \quad F_{n2} -= f_{12}.$$

Different OpenMP threads can therefore write to the same force and virial entries at the same time.

The implemented solution uses thread-local accumulation buffers:

1. Each OpenMP thread allocates private force and virial arrays.
2. The outer atom loop is parallelized with `#pragma omp for`.
3. Pair contributions are accumulated only into the calling thread's private arrays.
4. After the parallel loop, private arrays are reduced into the shared output arrays inside a short critical section.

This keeps the force calculation race-free while preserving the original pairwise force and virial formulas.

3 Correctness Test

A small standalone NEP checker was added:

- Source: `source/source_md/tools/nep_omp_check.cpp`
- Runner: `source/source_md/tools/run_nep_omp_check.sh`
- Model: `tests/PP_ORB/nep_hfo2.txt`

The checker computes the same six-atom Hf/O configuration using one OpenMP thread as the reference, then repeats the calculation with multiple thread counts. It reports average runtime and maximum absolute differences for potential, force, and virial arrays.

4 Initial Run Results

The following command was used on the local machine:

```
tools/run_nep_omp_check.sh ../../tests/PP_ORB/nep_hfo2.txt 1,2,4 100
```

The compiler selected by the script was `g++-15` with `-O2 -fopenmp`. The results were:

Threads	Avg. time (ms)	Speedup	Max ΔE	Max ΔF	Max ΔV
1	0.96246	1.00	0	0	0
2	0.78283	1.23	0	2.22045×10^{-16}	8.88178×10^{-16}
4	0.80320	1.20	0	4.44089×10^{-16}	1.33227×10^{-15}

Table 1: Initial NEP OpenMP correctness and runtime check.

5 Analysis

The force and virial differences are at floating-point roundoff level, so the OpenMP force path is numerically consistent with the single-thread reference. The two-thread and four-thread runs show modest speedups of about $1.23\times$ and $1.20\times$ for this small test case. The four-thread result is slightly slower than the two-thread result, which is reasonable for such a small system because thread startup, private buffer allocation, and final reduction overhead can dominate the actual NEP work.

For larger systems, the per-atom NEP descriptor and force work should become large enough for the OpenMP parallel region to amortize its overhead more effectively. Therefore, the current result should be interpreted mainly as a correctness and initial-performance validation, not as the final scalability limit.

6 How to Reproduce

```
cd source/source_md
tools/run_nep_omp_check.sh
```

Optional arguments are:

```
tools/run_nep_omp_check.sh MODEL_FILE THREAD_LIST REPEATS
tools/run_nep_omp_check.sh /path/to/nep.txt 1,2,4,8 50
```